

NORTHSTAR DATA ANALYST

System Architecture & Technical Documentation

CSV-based Natural Language Data Analysis Platform

Northstar is a local-first AI-powered data analysis application that lets users upload one or more CSV datasets, inspect data quality and schema, ask natural-language questions, and receive verified analytical results, charts, reports, and human-readable explanations. The LLM is used for understanding and generation, while Pandas and DuckDB perform the actual data processing and verification.

1. System Overview

- 1. Create a private workspace.
- 2. Upload one or more CSV files.
- 3. Profile datasets for schema, types, nulls, duplicates, unique values, samples, and statistics.
- 4. Ask questions in natural language.
- 5. Check whether the question is relevant to the selected dataset.
- 6. Classify the analytical intent and generate a safe SQL plan.
- 7. Validate generated SQL before execution.
- 8. Execute validated SQL against the uploaded data using DuckDB.
- 9. Use the verified result as evidence for the final explanation and visualization.
- 10. Render the answer, chart, SQL, verified rows, reports, and metadata in the React dashboard.

2. Technology Stack

Layer	Technology	Responsibility
Frontend	React 19, TypeScript, Vite	Workspace UI and result rendering
Visualization	Recharts, Lucide React	Charts and UI icons
Backend	Python, FastAPI, Uvicorn	REST API and orchestration
Data Processing	Pandas	CSV loading, profiling, deterministic analysis
Query Engine	DuckDB	Read-only SQL execution over in-memory DataFrames
Metadata Storage	SQLite + SQLAlchemy	Users, projects, datasets, messages
Authentication	Authlib, httpx, Starlette sessions	Google OAuth and session handling
LLM	Groq OpenAI-compatible API	Intent, SQL generation, answer refinement

3. High-Level Architecture

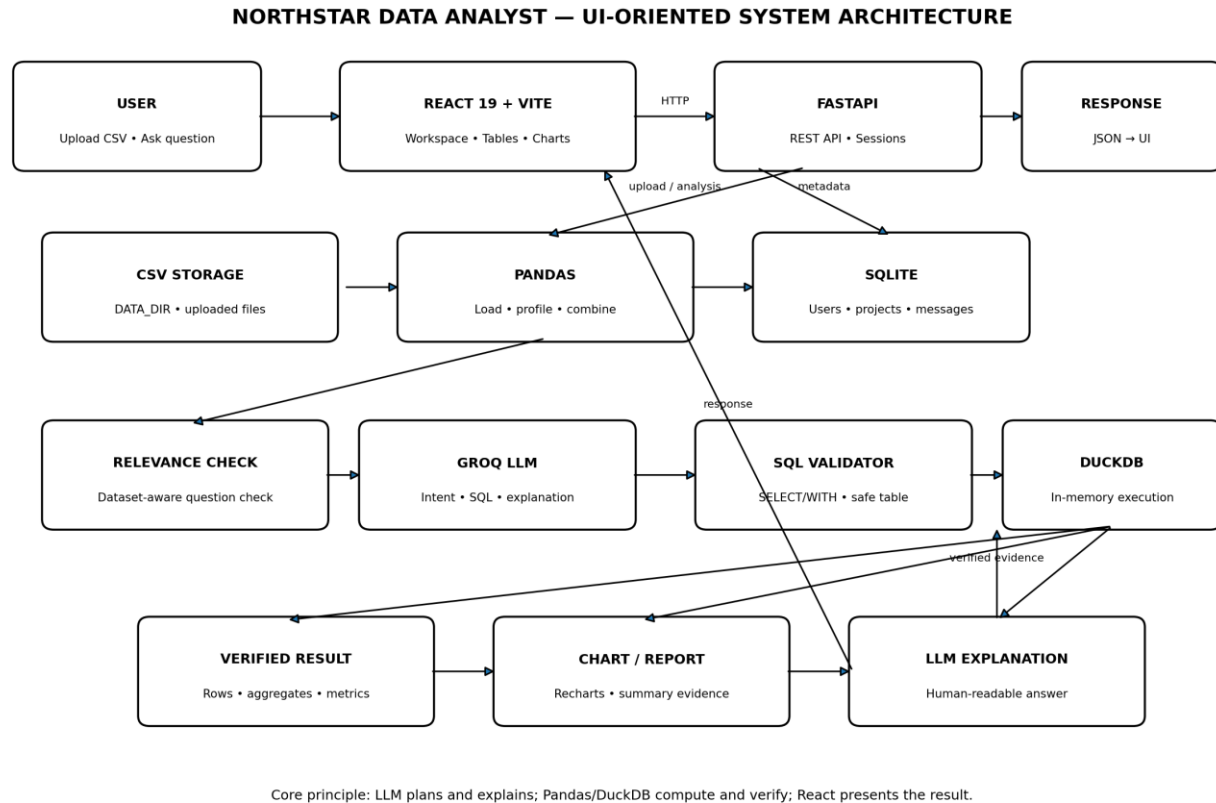


Figure 1. UI-oriented Northstar architecture.

The architecture is deliberately separated into presentation, API/orchestration, data processing, AI assistance, validation, execution, and persistence responsibilities. This separation prevents the LLM from becoming the source of truth for numerical results.

4. Component Responsibilities

React + Vite: Provides the workspace UI, upload interface, dataset inspection, question input, conversation history, tables, charts, SQL/result display, and reports.

FastAPI: Receives HTTP requests, manages authentication/session context, handles uploads, routes analysis requests, and returns structured JSON responses.

SQLite + SQLAlchemy: Stores users, workspaces/projects, dataset metadata, and conversation messages. Analytical CSV data is not treated as the primary SQLite dataset.

CSV Storage: Stores uploaded CSV files under the configured DATA_DIR.

Pandas: Loads CSV files, profiles data, combines compatible files, performs deterministic analysis, and prepares evidence/chart data.

Relevance Check: Determines whether the user's question can be answered from the selected dataset rather than allowing unrelated questions to proceed.

Groq LLM: Classifies analytical intent, proposes SQL, and refines the final verified result into a concise human-readable answer.

SQL Validator: Treats LLM SQL as untrusted input and permits only a single read-only SELECT/WITH statement against the registered dataset table.

DuckDB: Compiles and executes validated SQL in memory against the uploaded Pandas DataFrame.

Chart/Report Layer: Uses verified analytical evidence to construct supported charts and structured summary reports.

5. End-to-End Question Flow

1. User enters a natural-language question.
2. Frontend sends the question and selected dataset ID(s) to the analysis endpoint.
3. Backend loads the selected CSV files with Pandas.
4. Files are combined according to the supported multi-file rules.
5. The relevance layer checks whether the question is related to the dataset.
6. Schema information and the question are supplied to the intent classifier.
7. The intent layer determines the analysis type and produces a deterministic fallback shape/evidence.
8. Groq may generate SQL using the question, schema, intent, and fallback shape.
9. The SQL validator checks the generated statement.
10. DuckDB executes only the validated read-only SQL.
11. The resulting rows/aggregates are treated as verified evidence.
12. The evidence is passed to the answer-refinement stage.
13. The API returns the answer, verified rows, SQL, chart/report data, and metadata.
14. React renders the final result.

6. Core Data/AI Flow

The central design rule is:

LLM plans and explains → Pandas/DuckDB compute and verify → React presents

Conceptual flow:

User Question → Dataset Schema → Intent Classification → SQL Generation → SQL Validation → DuckDB Execution → Verified Result → LLM Explanation → React UI

7. SQL Safety Architecture

- LLM output is treated as untrusted input.

- Only one SQL statement is accepted.
- The statement must begin with SELECT or WITH.
- Write operations are rejected.
- Extension and dangerous operations are rejected.
- Only the registered dataset table may be referenced.
- DuckDB EXPLAIN/compilation is used before execution.
- Execution happens in memory against the uploaded DataFrame.

8. Multi-File Analysis

- Multiple CSV files can be uploaded in one operation.
- Matching schemas are concatenated.
- Files with common non-numeric columns can be merged on those common columns.
- Incompatible files can be combined with missing columns represented as null.
- The UI can analyze all uploaded files or a selected dataset.
- Valid files are retained when another file in the same upload fails validation.

9. Summary Report Flow

For overview questions such as “Give me a complete summary of this dataset”, Northstar produces:

- Row and column counts
- Duplicate row count
- Missing cell count
- Per-column schema and null statistics
- Unique values and sample values
- Numeric count, mean, median, minimum, maximum, and standard deviation
- Top categorical values and frequencies
- First 10 sample rows
- A read-only DuckDB row-count verification query

10. Storage Model

Northstar's current implementation uses SQLite rather than PostgreSQL or Redis.

- SQLite: users, projects, dataset metadata, and conversation messages.
- DATA_DIR: uploaded CSV files.
- Pandas: in-memory data processing.
- DuckDB: in-memory analytical SQL execution.

11. API Surface

Method	Endpoint	Purpose
GET	/health	Backend health check
GET	/api/auth/me	Current session
GET	/api/projects	List workspaces
POST	/api/projects	Create a workspace
GET	/api/projects/{id}	Workspace and dataset metadata
POST	/api/projects/{id}/datasets	Upload one or more CSV files

POST	/api/projects/{id}/datasets/{dataset_id}/ask	Analyze a question
GET	/api/projects/{id}/datasets/{dataset_id}/rows	Paginated dataset rows
GET	/api/projects/{id}/messages	Conversation history

12. Security and Validation

- Google OAuth state is checked before accepting callbacks.
- Session cookies are managed by Starlette middleware.
- Uploaded files are restricted to CSV and size-limited.
- Generated SQL is validated before execution.
- Only read-only SELECT/WITH statements are permitted.
- LLM output is never trusted as executable code.

13. Project Structure

```

Northstar Data Analyst/
├── backend/
│   ├── app/
│   │   ├── api/routes.py
│   │   ├── auth.py
│   │   ├── config.py
│   │   ├── db.py
│   │   ├── main.py
│   │   └── services/
│   │       ├── analysis.py
│   │       ├── anomalies.py
│   │       ├── intent.py
│   │       ├── llm.py
│   │       ├── profiling.py
│   │       └── query.py
│   ├── tests/
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── App.tsx
│   │   ├── api.ts
│   │   └── types.ts
│   └── package.json
└── .env.example

```

14. Failure and Fallback Strategy

Groq is optional. If the API key is missing or an LLM request fails, the deterministic Pandas analysis and safe fallback SQL path can keep core analysis available. This makes the application less dependent on a single external AI service.

15. Important Design Principle for Presentation

When explaining Northstar in an interview or project review, emphasize that the LLM does not directly calculate or decide the truth of a numerical answer. It interprets the question and assists with query generation; the validated SQL is executed against the actual uploaded dataset, and the resulting data becomes the evidence used for the final response.

16. Architecture Diagram in Mermaid

flowchart TD

```
U[User] --> F[React + Vite]
F --> A[FastAPI REST API]

A --> AUTH[Session Authentication / Google OAuth]
A --> STORE[(SQLite)]
A --> UPLOAD[CSV Upload]
UPLOAD --> PROFILE[Pandas Profiling]
PROFILE --> FILES[(CSV Files / DATA_DIR)]

F --> Q[Question + Dataset IDs]
Q --> LOAD[Pandas Load]
LOAD --> MERGE[Multi-file Combine / Merge]
MERGE --> REL[Dataset Relevance Check]

REL -->|Unrelated| MSG[Dataset-related message]
REL -->|Related| SCHEMA[Schema + Question]
SCHEMA --> INTENT[Groq Intent Classification]
INTENT --> PLAN[Analysis Intent]
PLAN --> DET[Deterministic Analysis]
PLAN --> SQLGEN[Groq SQL Generation]
DET --> FALLBACK[Safe Fallback SQL]
SQLGEN --> SQL[Generated SQL]
FALLBACK --> SQL

SQL --> VALIDATE[SQL Safety Validation]
VALIDATE -->|SELECT/WITH only| DB[DuckDB In-Memory]
DB --> VERIFIED[Verified Result]

DET --> EVIDENCE[Evidence / Chart / Summary]
VERIFIED --> EXPLAIN[Groq Answer Refinement]
EVIDENCE --> EXPLAIN

EXPLAIN --> RESPONSE[FastAPI Response]
VERIFIED --> RESPONSE
RESPONSE --> F
F --> UI[Answer + Charts + SQL + Verified Rows + Report]
```

17. Documentation Basis

This document is based on the supplied Northstar project documentation, including its current SQLite storage model, Pandas/DuckDB analytical pipeline, FastAPI backend, React frontend, and optional Groq integration.